

Apprentissage profond pour la détection d'anomalies dans les données de journalisation : une revue

Max Landauer, Sebastian Onder, Florian Skopik et Markus Wurzenberger

Résumé

L'analyse automatique des fichiers journaux permet la détection précoce d'incidents importants, tels que les défaillances de systèmes. En particulier, les techniques de détection d'anomalies auto-apprenantes capturent les motifs présents dans les données de journalisation et signalent ensuite aux opérateurs système les occurrences inattendues d'événements de logs, sans qu'il soit nécessaire de fournir ou de modéliser manuellement à l'avance des scénarios anormaux.

Récemment, un nombre croissant d'approches exploitant les réseaux neuronaux d'apprentissage profond ont été proposées à cette fin. Ces approches ont démontré des performances de détection supérieures à celles des techniques classiques d'apprentissage automatique, tout en résolvant certains problèmes liés à l'instabilité des formats de données.

Cependant, il existe de nombreuses architectures différentes d'apprentissage profond, et il n'est pas trivial d'encoder des données de logs brutes et non structurées afin qu'elles puissent être analysées par des réseaux neuronaux. C'est pourquoi les auteurs réalisent une revue systématique de la littérature, qui fournit une vue d'ensemble des modèles utilisés, des mécanismes de prétraitement des données, des techniques de détection d'anomalies et des méthodes d'évaluation.

Cette revue ne compare pas quantitativement les approches existantes. Elle vise plutôt à aider les lecteurs à comprendre les aspects pertinents des différentes architectures de modèles et met en évidence les questions ouvertes pour les travaux futurs.

Mots-clés : données de journalisation, détection d'anomalies, réseaux neuronaux, apprentissage profond.

I. Introduction

Les fichiers journaux constituent une source très riche d'informations pour la surveillance des systèmes informatiques. La majorité des événements de logs sont généralement générés comme conséquence d'opérations normales du système, telles que le démarrage et l'arrêt de processus, le redémarrage de machines virtuelles, l'accès des utilisateurs aux ressources, etc.

Cependant, les applications produisent également des logs lorsque des états système défectueux ou autrement indésirables se produisent. Il peut s'agir, par exemple, de processus ayant échoué, de problèmes de disponibilité ou d'incidents de sécurité. Ces traces d'activités inattendues, et potentiellement dangereuses, sont importantes pour les opérateurs système, qui doivent agir rapidement afin de prévenir ou de réduire les dommages causés au système et d'éviter des effets négatifs en cascade.

Le principal problème de ce type d'analyse des fichiers journaux est qu'il n'est pas simple d'identifier ces événements pertinents parmi le nombre beaucoup plus important de traces moins intéressantes correspondant à une utilisation normale du système. En particulier, la quantité massive de logs produits par les applications modernes rend l'analyse manuelle impossible et impose le recours à des mécanismes automatiques.

Malheureusement, les signatures et règles codées manuellement, qui recherchent des mots-clés spécifiques dans les logs, n'ont qu'une applicabilité limitée. Elles ne conviennent pas aux scénarios qui ne sont pas connus à l'avance. Il est donc nécessaire de déployer des techniques de détection d'anomalies capables d'apprendre automatiquement des modèles représentant le comportement normal du système, puis de révéler toute déviation par rapport à ces modèles comme une activité potentiellement défavorable nécessitant l'attention d'opérateurs humains.

L'apprentissage automatique fournit de nombreuses techniques utilisables pour la détection d'anomalies dans les fichiers journaux. De nombreuses approches ont déjà été proposées, notamment le clustering, l'extraction de workflows, l'analyse statistique des paramètres d'événements, l'analyse de séries temporelles pour reconnaître les changements de fréquence d'événements, ainsi que plusieurs autres méthodes.

Récemment, les chercheurs ont commencé à utiliser des réseaux neuronaux profonds pour la détection d'anomalies basée sur les logs, dans le but de reproduire les succès de l'apprentissage profond observés dans la reconnaissance d'images et de la parole, domaines dans lesquels il surpasse les méthodes classiques d'apprentissage automatique.

Cependant, les événements de logs système sont généralement non structurés et impliquent des dépendances complexes. Il n'est donc pas trivial de préparer les données de manière à permettre leur ingestion par des réseaux neuronaux et d'en extraire des caractéristiques pertinentes pour la détection. De plus, la grande variété d'architectures d'apprentissage profond existantes, telles que les réseaux neuronaux récurrents ou convolutionnels, rend difficile le choix d'un modèle approprié pour un cas d'usage donné, ainsi que la compréhension de ses exigences concernant le format et les propriétés des données d'entrée.

À la connaissance des auteurs, il n'existe actuellement qu'un aperçu limité de l'état de l'art concernant la détection d'anomalies basée sur les logs au moyen de l'apprentissage profond. Par conséquent, il est difficile de comprendre quelles caractéristiques peuvent être extraites des données brutes de logs, comment ces caractéristiques peuvent être transformées dans un format adapté à l'ingestion par des réseaux neuronaux, et quelles architectures de modèles conviennent à la détection de motifs spécifiques dans les logs.

Les revues existantes ne comparent qu'un petit nombre d'approches de détection d'anomalies et se concentrent principalement sur les motifs séquentiels dans les données de logs. D'autres études présentent de larges analyses sur les données de logs système, mais ne couvrent pas suffisamment les modèles d'apprentissage profond et leurs défis. Certaines se concentrent plutôt sur le trafic réseau que sur les données de logs système.

Les auteurs réalisent donc une revue systématique de la littérature sur l'apprentissage profond appliqué à la détection d'anomalies dans les données de journalisation. Leur objectif principal est d'étudier les publications scientifiques portant sur les architectures de modèles utilisées, leurs exigences et transformations respectives pour traiter des logs non structurés, les méthodes employées pour distinguer les échantillons normaux des échantillons anormaux, ainsi que les évaluations présentées.

Les résultats de cette étude sont utiles à la fois pour les chercheurs et pour l'industrie. En effet, une meilleure compréhension des défis et des caractéristiques des différents algorithmes d'apprentissage profond permet d'éviter certains pièges lors du développement de techniques de détection d'anomalies. Elle facilite également la sélection de systèmes de détection existants, aussi bien pour les cas d'usage académiques que réels.

En outre, une étude détaillée des stratégies de prétraitement est essentielle pour exploiter toute l'information disponible dans les logs lors de la détection d'anomalies. Elle permet également de comprendre l'influence des représentations de données sur les capacités de détection, notamment quels types d'anomalies peuvent être détectés et dans quelles conditions.

Concernant les évaluations scientifiques, les auteurs accordent une attention particulière aux aspects pertinents de la conception expérimentale, notamment les jeux de données, les métriques et la reproductibilité. Leur objectif est de mettre en évidence les insuffisances des stratégies d'évaluation couramment utilisées et de proposer des pistes d'amélioration.

Enfin, cette étude vise également à constituer un travail de référence et un point de départ pour les recherches futures. Les auteurs précisent que cette revue ne compare pas quantitativement les performances de détection des approches analysées, car

seules quelques implémentations open source sont disponibles et des comparaisons de ces approches ont déjà été présentées dans d'autres revues.

Conformément à ces objectifs, les auteurs formulent les questions de recherche suivantes :

RQ1 : Quels sont les principaux défis de la détection d'anomalies basée sur les logs avec l'apprentissage profond ?

RQ2 : Quels algorithmes d'apprentissage profond de l'état de l'art sont généralement appliqués ?

RQ3 : Comment les données de logs sont-elles prétraitées afin d'être ingérées par des modèles d'apprentissage profond ?

RQ4 : Quels types d'anomalies sont détectés et comment sont-elles identifiées comme telles ?

RQ5 : Comment les modèles proposés sont-ils évalués ?

RQ6 : Dans quelle mesure les approches dépendent-elles de données étiquetées et prennent-elles en charge l'apprentissage incrémental ?

RQ7 : Dans quelle mesure les résultats présentés sont-ils reproductibles en termes de disponibilité du code source et des données utilisées ?

II. Contexte

Dans cette section, les auteurs clarifient d'abord certains concepts et termes généraux importants pour la détection d'anomalies dans les données de journalisation à l'aide de l'apprentissage profond. Ils présentent ensuite les défis scientifiques propres à ce domaine de recherche.

A. Définitions préliminaires

L'étude réalisée dans cet article repose sur la compréhension de trois concepts principaux : **l'apprentissage profond, les données de journalisation et la détection d'anomalies**. Toutefois, les caractéristiques exactes et les exigences associées à ces différents domaines peuvent varier selon les champs de recherche et selon la littérature existante.

Les auteurs décrivent donc ci-dessous les propriétés fondamentales de ces trois concepts.

1. Apprentissage profond

Les réseaux neuronaux artificiels ont été développés dans le but de reproduire, sous forme de nœuds de communication interconnectés, certains mécanismes des systèmes biologiques de traitement de l'information.

Pour cela, un nombre variable de nœuds est organisé en plusieurs couches successives. On distingue généralement :

- une **couche d'entrée**, qui reçoit les données ;
- plusieurs **couches cachées**, qui relient les couches voisines au moyen de connexions pondérées ;
- une **couche de sortie**, qui produit le résultat final.

Les nœuds s'activent lorsqu'ils reçoivent certains signaux sur leurs connexions. Cette activation génère ensuite l'entrée destinée aux couches suivantes.

L'idée principale est que ces réseaux sont capables de reconnaître des structures non linéaires dans les données d'entrée, puis de classer les instances traitées grâce à un processus d'entraînement. Cet entraînement consiste à minimiser les erreurs de classification en ajustant progressivement les poids des connexions.

Les réseaux neuronaux artificiels permettent plusieurs formes d'apprentissage :

Apprentissage supervisé : les étiquettes de toutes les classes sont disponibles. Par exemple, les données peuvent être marquées comme normales ou anormales.

Apprentissage semi-supervisé : seules certaines classes disposent d'étiquettes.

Apprentissage non supervisé : aucune étiquette n'est disponible.

De manière générale, les algorithmes d'apprentissage profond sont compris comme des réseaux neuronaux comportant plusieurs couches cachées. Plusieurs architectures de réseaux neuronaux profonds ont été proposées, par exemple les **réseaux neuronaux récurrents**, particulièrement adaptés aux données séquentielles.

L'apprentissage profond a démontré sa capacité à surpasser les méthodes classiques d'apprentissage automatique, comme les machines à vecteurs de support ou les arbres de décision, et même parfois les experts humains dans de nombreux domaines d'application, notamment la classification d'images, la reconnaissance vocale et d'autres tâches complexes.

2. Données de journalisation

Les données de journalisation correspondent à une séquence chronologique d'événements, pouvant être composés d'une seule ligne ou de plusieurs lignes. Ces événements sont générés par des applications afin de conserver une trace permanente de certains états du système, notamment pour faciliter l'analyse forensic manuelle en cas de panne ou d'incident inattendu.

Les événements de logs sont généralement disponibles sous forme textuelle. Ils peuvent prendre différentes formes :

- des vecteurs structurés, par exemple des valeurs séparées par des virgules ;
- des objets semi-structurés, par exemple des paires clé-valeur ;
- des messages non structurés, lisibles par l'être humain, mais contenant des types d'événements hétérogènes.

Il n'existe pas de format unique et universel pour les logs. Cependant, les événements de journalisation contiennent généralement un horodatage indiquant le moment où ils ont été générés.

D'autres paramètres peuvent parfois être présents selon le type de données de logs, par exemple :

- le niveau de journalisation, comme INFO ou ERROR ;
- l'identifiant du processus ;
- des informations permettant de relier plusieurs événements appartenant à une même séquence.

Un événement de log isolé décrit donc une partie de l'état du système à un instant donné. En revanche, un groupe d'événements de logs peut représenter les workflows dynamiques de la logique applicative sous-jacente.

Cela s'explique par le fait que les événements de logs sont produits par des instructions d'affichage ou d'enregistrement placées volontairement par les développeurs dans leur code. Ces instructions servent à comprendre les activités du programme et à faciliter le débogage.

Ces instructions comportent généralement deux parties :

Une partie statique : chaînes de caractères écrites directement dans le code.

Une partie variable : paramètres déterminés dynamiquement pendant l'exécution du programme.

Dans les années précédentes, une grande quantité de recherches a porté sur l'extraction automatique de ce que l'on appelle les **log keys**, également appelées signatures de logs, modèles de logs ou templates de logs. Ces éléments représentent

les modèles des instructions originales ayant produit les logs et permettent de les analyser automatiquement.

Ces parseurs permettent de dériver des valeurs plus exploitables que les messages de logs bruts et non structurés. Ils permettent notamment :

1. d'attribuer des identifiants de type d'événement aux lignes de logs ;
2. d'extraire les paramètres contenus dans les messages de logs.

3. Détection d'anomalies

Les anomalies sont les instances d'un jeu de données qui présentent des caractéristiques rares, inattendues ou différentes du comportement général des autres données. Elles se distinguent donc du reste de l'ensemble.

Pour détecter ces anomalies, on mesure généralement la conformité des instances à travers un ou plusieurs attributs continus ou catégoriels associés à chaque instance. Ces attributs permettent ensuite de calculer des métriques de similarité.

Lorsque les données sont indépendantes, il peut suffire d'identifier les points ou petits groupes de points très différents des autres données. Ces cas sont appelés **anomalies ponctuelles**.

Cependant, dans les données où les instances ne sont pas indépendantes les unes des autres, par exemple les données ordonnées comme les logs, deux autres types d'anomalies apparaissent.

Anomalies contextuelles

Les anomalies contextuelles sont des instances qui ne sont considérées comme anormales qu'en fonction du contexte dans lequel elles apparaissent.

Par exemple, le démarrage d'une procédure de sauvegarde quotidienne peut être normal en soi. Mais si cette sauvegarde se lance soudainement en dehors de l'horaire prévu, l'événement peut devenir anormal.

Anomalies collectives

Les anomalies collectives sont des groupes d'instances qui deviennent anormaux uniquement par leur occurrence combinée. Chaque événement pris individuellement peut sembler normal, mais leur combinaison ou leur séquence peut révéler un comportement anormal.

C'est le cas, par exemple, d'une séquence particulière d'événements de logs.

Une hypothèse implicite de la plupart des techniques de détection d'anomalies est que les données analysées contiennent beaucoup moins d'anomalies que d'instances normales. Cette hypothèse permet parfois de réaliser la détection de manière totalement non supervisée, c'est-à-dire sans données étiquetées pour entraîner les modèles.

Cependant, de nombreuses approches scientifiques utilisent plutôt une détection semi-supervisée. Dans ce cas, les données d'entraînement ne contiennent que des instances normales, tandis que l'évaluation est réalisée sur un jeu de test contenant à la fois des instances normales et anormales.

Le principal avantage du fonctionnement semi-supervisé est que les instances anormales ne sont pas apprises par le modèle. De plus, il est souvent relativement simple de collecter des données normales, alors que la modélisation et l'étiquetage des anomalies sont beaucoup plus difficiles.

Par conséquent, les approches de détection d'anomalies supervisées ont une applicabilité plus faible et sont relativement rares.

B. Défis

La détection d'anomalies basée sur les logs est un domaine de recherche actif depuis plusieurs décennies. La plupart des approches proposées reposaient auparavant sur des techniques classiques d'apprentissage automatique.

Cependant, les dernières années ont vu une forte augmentation des approches utilisant l'apprentissage profond pour détecter des événements de logs anormaux liés à des comportements inattendus du système.

Les auteurs résument les principaux défis à relever pour obtenir une détection efficace et applicable.

1. Représentation des données

Les systèmes d'apprentissage profond utilisent généralement des données d'entrée structurées et numériques.

Or, il n'est pas simple d'introduire des données de logs dans des réseaux neuronaux, car ces données mélangent souvent :

- des types d'événements hétérogènes ;
- des messages non structurés ;

- des paramètres catégoriels.

Le premier défi est donc de transformer des logs textuels bruts en représentations numériques exploitables par un modèle d'apprentissage profond.

2. Instabilité des données

À mesure que les applications évoluent, de nouveaux types d'événements de logs peuvent apparaître. Ces nouveaux événements peuvent différer de ceux observés dans les données d'entraînement.

De plus, les comportements du système changent avec le temps, car les environnements technologiques et les modes d'utilisation évoluent.

Les systèmes d'apprentissage profond doivent donc pouvoir mettre à jour leurs modèles de manière incrémentale et adapter leur référence du comportement normal du système afin de permettre une détection en temps réel.

3. Déséquilibre des classes

La détection d'anomalies suppose naturellement que les événements normaux sont beaucoup plus nombreux que les événements anormaux.

Or, de nombreuses approches basées sur les réseaux neuronaux peuvent avoir des performances sous-optimales lorsque les jeux de données sont très déséquilibrés.

Cela signifie que le modèle peut apprendre trop fortement la classe normale et ne pas détecter correctement les rares événements anormaux.

4. Diversité des manifestations anormales

Les anomalies peuvent affecter les événements de logs et leurs paramètres de nombreuses manières.

Elles peuvent apparaître sous forme de changements dans :

- les motifs séquentiels ;
- les fréquences d'événements ;
- les corrélations ;
- les temps d'arrivée entre événements ;

- les paramètres associés aux logs.

Les techniques de détection sont souvent conçues pour reconnaître certaines propriétés spécifiques d'un type d'anomalie. Elles ne sont donc pas toujours applicables de manière générale.

5. Disponibilité des étiquettes

Les anomalies représentent des comportements inattendus du système. En général, il n'existe donc pas d'exemples d'anomalies étiquetées disponibles pour l'entraînement.

Cela limite l'utilisation des systèmes d'apprentissage profond à des approches semi-supervisées ou non supervisées. Or, ces approches ont généralement des performances de détection plus faibles que les méthodes supervisées.

6. Traitement en flux

Les logs sont générés comme un flux continu de données.

Pour permettre une surveillance en temps réel, et non seulement une analyse forensic après incident, les systèmes d'apprentissage profond doivent être conçus pour traiter les données en un seul passage.

Cela concerne à la fois la détection et la mise à jour du modèle.

7. Volume des données

Les logs sont produits en très grands volumes. Certains systèmes génèrent des millions, voire des milliards d'événements chaque jour.

Il faut donc des algorithmes efficaces pour garantir un traitement en temps réel dans des applications pratiques, en particulier lorsque les systèmes fonctionnent sur des machines disposant de peu de ressources de calcul, comme les dispositifs en périphérie réseau.

8. Entrelacement des logs

Lorsque de nombreux processus fonctionnent simultanément, ou lorsque des logs distribués sont collectés de manière centralisée, les séquences d'événements liés peuvent être mélangées les unes aux autres.

Il devient alors difficile de reconstruire les séquences originales d'événements, surtout lorsque les logs ne contiennent pas d'identifiants de session.

9. Qualité des données

Une faible qualité des données peut résulter d'une mauvaise collecte des logs ou de problèmes techniques pendant leur génération.

Cela peut avoir des effets négatifs sur l'efficacité de l'apprentissage automatique.

Les problèmes courants incluent :

- des horodatages incorrects ;
 - un mauvais ordre des événements ;
 - des événements manquants ;
 - des enregistrements dupliqués ;
 - des événements mal étiquetés.
-

10. Explicabilité du modèle

Les approches fondées sur les réseaux neuronaux souffrent généralement d'une plus faible explicabilité que les méthodes classiques d'apprentissage automatique.

Il est souvent difficile de comprendre pourquoi un modèle a produit une classification correcte ou incorrecte.

Cette difficulté est particulièrement problématique lorsqu'il faut prendre des décisions justifiées face à un comportement critique du système ou à un incident de sécurité.

La section suivante de l'article est **III. Survey Method / Méthode de revue**, où les auteurs expliquent comment ils ont sélectionné les publications scientifiques analysées.

III. Méthode de revue

Cette section présente la méthode utilisée pour réaliser la revue systématique de la littérature. Les auteurs décrivent d'abord leur stratégie de collecte des publications pertinentes, puis les critères d'évaluation employés pour analyser les articles retenus.

A. Stratégie de recherche

Dans cette partie, les auteurs expliquent le processus utilisé pour rassembler les publications à inclure dans l'étude.

La démarche se fait en deux grandes étapes :

1. une première collection de publications est constituée au moyen d'une recherche en ligne ;
2. les articles pertinents sont ensuite sélectionnés à l'aide de critères d'inclusion, d'exclusion et de qualité.

1. Collecte initiale de la littérature avec une chaîne de recherche

Pour obtenir un premier ensemble d'approches représentant l'état de l'art, les auteurs construisent une **chaîne de recherche** destinée à interroger des bases de données scientifiques courantes.

Cette chaîne de recherche est conçue de façon à retrouver uniquement les publications contenant les trois concepts principaux de cette revue :

- les données de journalisation ;
- la détection d'anomalies ;
- l'apprentissage profond.

Comme certaines publications utilisent des terminologies différentes, les auteurs ajoutent aussi plusieurs variantes afin de réduire le risque de manquer des articles importants.

Pour désigner les données de journalisation, ils utilisent notamment les termes suivants :

- log data ;
- system log(s) ;
- event log(s) ;
- log file(s) ;
- log event(s).

Pour désigner l'apprentissage profond, ils ajoutent aussi l'expression :

- neural network(s).

En revanche, ils évitent d'utiliser le mot log seul, car ce terme produit beaucoup de résultats liés aux logarithmes, qui ne sont pas pertinents pour l'étude.

La figure 1 de l'article présente donc la composition finale de cette chaîne de recherche. Elle combine les termes liés aux logs, à la détection d'anomalies et à l'apprentissage profond.

Les auteurs utilisent ensuite cette chaîne pour collecter des publications dans plusieurs bases scientifiques :

- ScienceDirect ;
- Scopus ;
- SpringerLink ;
- ACM Digital Library ;
- IEEE Xplore ;
- Google Scholar ;
- Web of Science.

La recherche a été menée en janvier 2022 et a produit un total de **2925 publications**.

2. Sélection des publications pertinentes

Afin d'éliminer les publications non pertinentes et de réduire l'ensemble initial à une taille raisonnable, les auteurs définissent plusieurs critères de sélection.

Le critère principal d'inclusion est le suivant :

Le modèle proposé dans la publication doit appliquer des techniques d'apprentissage profond, c'est-à-dire un réseau neuronal multicouche, pour la détection d'anomalies dans des données de logs hétérogènes et non structurées.

Les auteurs définissent ensuite plusieurs critères d'exclusion.

Une publication est exclue si :

- elle ne montre pas clairement que l'approche proposée est applicable ou conçue pour les données de logs ;
- une publication plus récente présente la même étude ou une étude très similaire ;
- elle applique simplement une approche existante sans modification nouvelle ;
- elle est écrite dans une langue autre que l'anglais ;
- elle n'est pas disponible sous forme électronique ;

- elle correspond à un livre, un rapport technique, une note de cours, une présentation, une revue ou une thèse.

Dans ces derniers cas, les auteurs privilégient, lorsque c'est possible, les publications correspondantes sous forme d'articles de conférence ou de journal.

Les auteurs ne limitent pas les publications à une période précise, afin de ne pas exclure d'anciens travaux qui pourraient rester pertinents. Toutefois, ils cherchent à éviter les publications de faible qualité scientifique.

Ils retiennent donc uniquement les publications qui satisfont aussi à des critères de qualité.

Une publication est retenue si :

- l'objectif de l'étude et ses résultats sont clairement énoncés ;
- les modèles d'apprentissage profond utilisés et leurs paramètres sont décrits de manière rigoureuse ;
- l'article inclut une évaluation solide de l'approche proposée ;
- les jeux de données utilisés pour l'évaluation sont référencés ou décrits ;
- les visualisations sont claires et lisibles.

L'évaluation doit notamment comprendre :

- une motivation convaincante du choix des expériences ;
- une description complète du processus d'évaluation ;
- une explication du choix des métriques ;
- une discussion détaillée des résultats obtenus et de leurs implications.

Le processus de sélection se déroule en deux étapes.

D'abord, les auteurs réduisent la collection initiale de publications à partir du titre et du résumé de chaque article, en utilisant les critères d'inclusion et d'exclusion. Après cette première étape, **331 publications** restent.

Ensuite, ils appliquent tous les critères mentionnés précédemment au contenu complet de chaque article. Au final, **62 articles** sont retenus pour la revue.

B. Caractéristiques étudiées

Pour analyser les publications retenues selon un schéma commun et répondre aux questions de recherche, les auteurs définissent une liste de caractéristiques à examiner pour chaque article.

Ces caractéristiques sont regroupées en plusieurs catégories.

1. Modèle d'apprentissage profond et mode de fonctionnement

Les premières questions portent sur le modèle d'apprentissage profond utilisé et son mode d'exploitation.

Les auteurs cherchent à répondre aux questions suivantes :

DL-1 : Quels modèles d'apprentissage profond sont utilisés ?

DL-2 : Quelles fonctions de perte sont appliquées pendant l'entraînement ?

DL-3 : L'approche prend-elle en charge l'apprentissage en ligne ou incrémental ?

DL-4 : L'entraînement se fait-il de manière non supervisée, semi-supervisée ou supervisée ?

Ces questions permettent de comprendre la nature des architectures employées, leur capacité d'adaptation et leur dépendance aux données étiquetées.

2. Prétraitement et transformation des logs

Les auteurs analysent ensuite les différentes manières dont les approches étudiées transforment les logs bruts en représentations numériques exploitables par des modèles d'apprentissage profond.

Ils posent les questions suivantes :

PP-1 : Comment les logs bruts sont-ils prétraités ?

PP-2 : Quelles caractéristiques sont extraites des logs prétraités ?

PP-3 : Comment les caractéristiques extraites sont-elles représentées sous forme de vecteurs ?

Cette partie est essentielle, car les réseaux neuronaux ne peuvent pas traiter directement des messages textuels non structurés. Les logs doivent donc être transformés en données numériques : vecteurs, matrices, embeddings ou représentations statistiques.

3. Détection d'anomalies

Le groupe suivant de questions concerne la manière dont les anomalies sont détectées.

Les auteurs veulent notamment comprendre si les types d'anomalies sont liés aux caractéristiques extraites des logs et à leur représentation vectorielle.

Les questions posées sont :

AD-1 : Quels types d'anomalies sont détectés par l'approche ?

AD-2 : Comment la sortie du réseau neuronal profond est-elle utilisée pour la détection d'anomalies ?

AD-3 : Comment les anomalies sont-elles distinguées des échantillons normaux ?

Cette partie permet de savoir si les approches détectent des anomalies ponctuelles, séquentielles, statistiques, fréquentielles ou d'autres formes de comportements anormaux.

4. Évaluation et reproductibilité

L'utilisation de jeux de données accessibles publiquement et la publication du code source constituent de bonnes pratiques scientifiques. Elles permettent aux autres chercheurs de vérifier les résultats présentés et de comparer différentes approches.

Le dernier groupe de questions porte donc sur l'évaluation et la reproductibilité.

Les questions sont les suivantes :

ER-1 : Quels jeux de données de logs sont utilisés pour évaluer l'approche ?

ER-2 : Quelles métriques d'évaluation sont utilisées ?

ER-3 : L'évaluation prend-elle en compte les performances d'exécution ?

ER-4 : Quelles approches sont utilisées comme références de comparaison ?

ER-5 : Les jeux de données utilisés sont-ils accessibles publiquement ?

ER-6 : Le code source de l'approche est-il accessible publiquement ?

Toutes ces questions sont évaluées individuellement pour chaque publication. Les résultats forment une matrice de caractéristiques présentée dans le tableau II de l'article. Cette matrice sert ensuite de base aux analyses et discussions.

IV. Résultats de la revue

Cette section présente les évaluations de toutes les publications étudiées en fonction des caractéristiques définies dans la section précédente.

Les auteurs commencent par fournir des informations générales sur les métadonnées des publications, puis analysent chaque caractéristique en détail.

A. Bibliométrie

Cette partie donne un aperçu de la répartition des publications selon leur année de publication et selon leur nombre de citations.

1. Publications par année

L'apprentissage profond appliqué à la détection d'anomalies dans les données de logs est un domaine de recherche relativement récent. Il a toutefois gagné en importance au cours des dernières années.

Ainsi, la majorité des publications de ce domaine ont été publiées pendant les deux ou trois années précédant l'étude.

La figure 2 de l'article montre la distribution des années de publication des articles analysés. Sur les **62 publications** retenues, **58** ont été publiées en **2019 ou après**.

Comme la recherche bibliographique a été menée au début de l'année 2022, seules deux publications de cette année-là sont incluses. Cependant, les auteurs s'attendent à ce que le nombre de publications augmente encore en 2022 et dans les années suivantes, conformément à la tendance observée dans le graphique.

2. Citations

Le nombre de citations est un indicateur couramment utilisé pour évaluer la pertinence et l'influence d'une publication scientifique.

Les auteurs présentent donc les six publications les plus citées selon Google Scholar.

L'article le plus cité est celui de **Du et al.**, publié en 2017, qui présente l'approche **DeepLog**. Cet article compte **963 citations** au moment de l'analyse.

DeepLog est considéré comme particulièrement influent, car il s'agit de l'une des premières approches fondées sur l'apprentissage profond pour détecter des séquences d'événements anormales dans les données de logs.

Plusieurs articles publiés par la suite s'appuient sur les fondements de DeepLog. Il est donc raisonnable de considérer que cet article a contribué à l'augmentation du nombre de publications dans ce domaine à partir de 2019.

Les auteurs notent aussi que l'article de **Yang et al.**, bien qu'antérieur à DeepLog, possède beaucoup moins de citations. La raison principale est que cet article se concentre sur l'analyse des tokens dans des événements de logs isolés, un sujet qui a reçu moins d'attention dans les recherches ultérieures que l'analyse des séquences d'événements.

Le tableau I de l'article présente les six publications les plus citées :

Approche	Année	Idée principale
DeepLog	2017	Détection et diagnostic d'anomalies à partir de logs système avec apprentissage profond
LogRobust	2019	Détection robuste d'anomalies dans des logs instables
LogAnomaly	2019	Détection non supervisée d'anomalies séquentielles et quantitatives
CNN sur logs Big Data	2018	Détection d'anomalies dans les logs de systèmes Big Data avec réseaux convolutionnels
Logsy	2020	Détection d'anomalies auto-attentive dans des logs non structurés
LogBERT	2021	Détection d'anomalies avec une architecture inspirée de BERT

IV.B. Techniques d'apprentissage profond

Cette section fournit une vue d'ensemble des propriétés des modèles d'apprentissage profond appliqués dans les publications examinées.

1. Modèles d'apprentissage profond

Il existe de nombreux types de modèles d'apprentissage profond pouvant être utilisés pour la détection d'anomalies dans les données de logs.

La forme la plus simple d'un réseau neuronal profond est le **perceptron multicouche**, ou **Multi-Layer Perceptron — MLP**. Dans ce type de réseau, toutes les couches sont entièrement connectées entre elles.

Cependant, à cause de leur simplicité, les MLP sont généralement moins performants que d'autres modèles spécialement conçus pour capturer les caractéristiques fréquentes dans les données séquentielles. Pour cette raison, ils apparaissent rarement seuls dans les travaux étudiés. Ils sont plutôt utilisés en combinaison avec d'autres modèles d'apprentissage profond ou comme mécanismes d'attention supplémentaires.

Réseaux neuronaux convolutionnels — CNN

Les **réseaux neuronaux convolutionnels**, ou **Convolutional Neural Networks — CNN**, étendent l'architecture des MLP en ajoutant des couches de convolution et de max-pooling parmi les couches cachées.

Ces couches permettent au réseau neuronal de capturer des caractéristiques plus abstraites dans les données d'entrée, tout en réduisant leurs dimensions.

Les CNN se sont révélés particulièrement efficaces pour la classification de données bidimensionnelles, notamment les images. Dans ce contexte, le réseau apprend des caractéristiques telles que des lignes ou des formes, indépendamment de leur position exacte dans l'image.

Dans le cas des logs, cette logique est adaptée en organisant les **log keys** dans une matrice. L'objectif est de permettre au réseau de capturer les relations entre événements, c'est-à-dire leurs dépendances temporelles.

Il existe aussi des variantes spécifiques de CNN, comme les **Temporal Convolutional Networks — TCN**. Ces réseaux sont conçus pour traiter des séries temporelles et capturer à la fois les dépendances à court terme et à long terme grâce aux convolutions causales dilatées.

Réseaux neuronaux récurrents — RNN

D'après le tableau II de l'article, les **réseaux neuronaux récurrents**, ou **Recurrent Neural Networks — RNN**, sont les architectures les plus utilisées dans les travaux étudiés.

Sur les **62 approches** analysées, **36** utilisent des RNN pour la détection d'anomalies.

La raison principale est que les RNN disposent de mécanismes de rétroaction qui leur permettent de conserver un état au fil du temps. Ils peuvent donc apprendre directement les motifs séquentiels d'exécution des événements dans les données d'entrée.

Or, ces motifs séquentiels sont précisément des indicateurs essentiels pour détecter les anomalies dans les jeux de données de logs couramment utilisés.

Plusieurs variantes de RNN ont été appliquées avec succès.

LSTM

L'une des architectures les plus répandues est le **Long Short-Term Memory** — **LSTM**.

Les LSTM sont conçus pour conserver des informations sur de longues périodes. Ils utilisent des cellules spécialisées comprenant notamment :

- des portes d'entrée ;
- des portes de sortie ;
- des portes d'oubli.

La plupart des approches utilisant des LSTM entraînent le réseau avec des séquences d'événements, ou avec des versions modifiées et enrichies de ces séquences. Ensuite, pendant la phase de test, le modèle signale les motifs séquentiels inhabituels comme des anomalies.

Bi-LSTM

Certaines approches utilisent des **Bi-LSTM**, c'est-à-dire des LSTM bidirectionnels.

Un Bi-LSTM est constitué de deux LSTM indépendants fonctionnant en parallèle :

- le premier traite la séquence dans l'ordre chronologique normal, du premier au dernier événement ;
- le second traite la séquence en sens inverse, du dernier événement vers le premier.

Cette architecture permet au modèle d'exploiter à la fois le contexte passé et le contexte futur d'une séquence.

Selon les expériences rapportées dans l'article, les Bi-LSTM peuvent surpasser les LSTM classiques dans certains cas.

GRU

Une autre variante populaire des RNN est le **Gated Recurrent Unit — GRU**.

Les GRU simplifient l'architecture interne des cellules récurrentes. Contrairement aux LSTM, ils n'utilisent principalement que deux types de portes :

- la porte de mise à jour ;
- la porte de réinitialisation.

Leur avantage principal est qu'ils sont plus efficaces sur le plan computationnel que les LSTM.

Cela devient important dans les cas où l'on veut déployer la détection d'anomalies sur des dispositifs ayant peu de ressources, par exemple des équipements en périphérie réseau ou des systèmes embarqués.

Autoencodeurs — AE

Les modèles décrits jusque-là sont surtout utilisés pour des problèmes de classification. Cependant, certains modèles d'apprentissage profond sont spécifiquement conçus pour fonctionner de manière non supervisée. Ils constituent donc un choix naturel pour la détection d'anomalies.

C'est le cas des **autoencodeurs**, ou **Autoencoders — AE**.

Un autoencodeur fonctionne en deux étapes :

1. un **encodeur** transforme les données d'entrée en une représentation compacte ;
2. un **décodeur** essaie ensuite de reconstruire les données d'origine à partir de cette représentation.

L'avantage est que ce processus ne nécessite pas forcément de données étiquetées.

L'idée principale est que le réseau apprend les caractéristiques dominantes des données normales, tout en négligeant le bruit. Ce mécanisme ressemble à certaines techniques de réduction de dimension, comme l'analyse en composantes principales.

Lorsqu'un nouvel échantillon est présenté à un autoencodeur déjà entraîné, si l'erreur de reconstruction est élevée, cela peut indiquer que l'échantillon est anormal.

Il existe plusieurs variantes d'autoencodeurs :

- les **Variational Autoencoders** — VAE, qui travaillent avec des distributions statistiques ;
 - les **Conditional Variational Autoencoders** — CVAE, qui ajoutent des informations conditionnelles, comme les types d'événements ;
 - les **Convolutional Autoencoders** — CAE, qui exploitent les avantages des CNN pour apprendre des caractéristiques indépendantes de leur position exacte.
-

Réseaux antagonistes génératifs — GAN

Les **Generative Adversarial Networks** — GAN constituent une autre approche d'apprentissage profond non supervisé.

Un GAN est composé de deux éléments qui sont entraînés en opposition :

1. un **générateur**, qui produit de nouvelles données ressemblant aux données d'entrée ;
2. un **discriminateur**, qui estime si une donnée provient réellement des données d'origine ou si elle a été générée artificiellement.

Le discriminateur aide progressivement le générateur à produire des données plus réalistes.

Dans le domaine de la détection d'anomalies dans les logs, différentes architectures ont été utilisées pour construire des GAN, notamment :

- des LSTM ;
 - des CNN combinés avec des GRU ;
 - des Transformers.
-

Transformers

Les **Transformers** utilisent des mécanismes dits de **self-attention**, ou auto-attention.

Leur objectif est d'intégrer les données dans un espace vectoriel où les éléments similaires sont proches les uns des autres, tandis que les éléments différents sont éloignés.

Le principe de l'attention consiste à attribuer des poids différents aux entrées selon leur contexte d'apparition. Par exemple, dans une phrase, certains mots sont plus importants que d'autres selon le sens global de la phrase.

C'est pour cette raison que les Transformers ont connu un grand succès dans le traitement automatique du langage naturel.

Dans les logs, l'idée est similaire : un événement de log ou un token peut avoir une importance différente selon les événements qui l'entourent.

Mécanismes d'attention

Les mécanismes d'attention n'apparaissent pas seulement dans les Transformers.

Ils sont aussi fréquemment utilisés pour améliorer les performances d'autres réseaux neuronaux profonds, notamment les RNN. Le principe est de donner un poids plus élevé aux entrées les plus pertinentes.

Cet effet est particulièrement utile lorsque les RNN doivent traiter de longues séquences. Dans ce cas, tous les événements d'une séquence n'ont pas nécessairement la même importance pour la détection d'une anomalie.

Les mécanismes d'attention peuvent être réalisés sous forme de réseaux entraînaables, par exemple des MLP.

Pour éviter la confusion avec les Transformers, les auteurs distinguent explicitement, dans le tableau II, les Transformers et les mécanismes d'attention ajoutés à d'autres architectures.

Graph Neural Networks — GNN

Les **Graph Neural Networks** — GNN sont beaucoup moins fréquents dans les travaux étudiés.

Les réseaux neuronaux classiques traitent généralement des données ordonnées :

- les CNN utilisent souvent des données bidimensionnelles ;
- les RNN utilisent des séquences d'observations.

Les GNN, en revanche, sont conçus pour traiter des graphes, c'est-à-dire des ensembles de sommets et d'arêtes.

Dans le cas des logs, une possibilité consiste à transformer les séquences d'événements en **graphes de sessions** :

- les sommets représentent les événements ;
- les arêtes représentent les transitions séquentielles entre ces événements.

L'article indique que Wan et al. sont les seuls auteurs étudiés à utiliser explicitement des GNN.

Evolving Granular Neural Network — EGNN

Un autre modèle moins courant est l'**Evolving Granular Neural Network — EGNN**.

Il s'agit d'un système d'inférence floue construit progressivement à partir de flux de données en ligne.

Ce type de modèle est intéressant parce qu'il peut s'adapter à des données qui changent avec le temps, ce qui correspond bien au problème des logs, dont la structure et les motifs peuvent évoluer lorsque les systèmes changent.

Combinaison de plusieurs modèles

La revue montre que beaucoup d'approches reposent sur un seul type de modèle d'apprentissage profond.

Cependant, certains auteurs combinent plusieurs modèles. Par exemple, une approche peut utiliser un MLP pour combiner la sortie d'un VAE avec celle d'un Transformer entraîné par apprentissage antagoniste.

Cela confirme un point important : dans la détection d'anomalies dans les logs, il n'existe pas une architecture universelle. Le choix du modèle dépend du type de logs, du type d'anomalies recherchées et de la représentation des données.

2. Fonctions de perte utilisées à l'entraînement

Les **fonctions de perte** sont essentielles dans l'entraînement des réseaux neuronaux profonds, car elles quantifient l'écart entre la sortie produite par le réseau et le résultat attendu.

La fonction de perte la plus fréquente dans les publications étudiées est la **Cross-Entropy**, notamment :

- la cross-entropy catégorielle pour les prédictions multi-classes ;
- la cross-entropy binaire pour distinguer simplement la classe normale de la classe anormale.

D'autres fonctions de perte courantes apparaissent aussi dans les travaux analysés.

Cross-Entropy

La **Cross-Entropy** mesure l'écart entre une vraie étiquette et l'étiquette prédite par le modèle.

Elle est très utilisée dans les problèmes de classification.

Dans le contexte des logs, elle peut servir par exemple à apprendre au modèle à prédire le prochain événement attendu dans une séquence. Si l'événement réel est très improbable selon le modèle, la séquence peut être considérée comme suspecte.

Fonction objectif Hyper-Sphere

La fonction **Hyper-Sphere Objective Function** mesure la distance entre un vecteur d'embedding et le centre d'une hypersphère.

L'idée est de représenter les données normales dans une région compacte de l'espace vectoriel. Lorsqu'un échantillon se situe loin du centre, il peut être considéré comme anormal.

Cette logique est adaptée à la détection d'anomalies, car elle cherche à modéliser une zone normale et à détecter les points qui s'en éloignent.

Mean Squared Error — MSE

Le **Mean Squared Error — MSE**, ou erreur quadratique moyenne, mesure l'écart moyen au carré entre les valeurs attendues et les valeurs prédites.

Il est souvent utilisé dans les tâches de régression et dans les autoencodeurs.

Dans un autoencodeur, si la reconstruction d'un log ou d'une séquence de logs produit une erreur élevée, cela peut indiquer que l'entrée ne ressemble pas aux données normales apprises pendant l'entraînement.

Divergence de Kullback-Leibler — KL

La **divergence de Kullback-Leibler** mesure l'écart entre deux distributions de probabilité.

Elle est particulièrement utile pour les modèles probabilistes, notamment les Variational Autoencoders.

Dans ce contexte, elle permet de comparer la distribution apprise par le modèle avec une distribution attendue.

Marginal Likelihood

La **vraisemblance marginale** est aussi utilisée dans certains modèles probabilistes.

Elle permet d'évaluer dans quelle mesure les données observées sont probables selon le modèle.

Plus une donnée est improbable, plus elle peut être considérée comme potentiellement anormale.

Fonctions de perte personnalisées

Certains articles utilisent des fonctions de perte personnalisées.

Ces fonctions peuvent combiner plusieurs objectifs, par exemple :

- minimiser l'erreur de classification ;
- rapprocher les données normales dans un espace vectoriel ;
- maximiser la séparation entre données normales et anormales ;
- améliorer l'apprentissage antagoniste dans les GAN.

L'article souligne aussi que **14 publications** ne précisent pas clairement la fonction de perte utilisée. C'est une limite importante, car la fonction de perte influence directement le comportement du modèle et la reproductibilité des résultats.

3. Mode de fonctionnement : apprentissage en ligne ou hors ligne

Dans les scénarios réels, toutes les données de logs ne sont pas disponibles à l'avance.

Les événements sont générés comme un flux continu et doivent idéalement être analysés au moment où ils apparaissent. Cela permet une détection presque en temps réel.

Cependant, la structure et les propriétés statistiques des logs peuvent varier avec le temps.

Par exemple :

- l'utilisation globale d'un système peut changer ;
- les interactions des utilisateurs peuvent évoluer ;
- l'environnement technologique peut être modifié ;
- de nouveaux événements de logs peuvent apparaître après une mise à jour logicielle ;
- certains templates de logs peuvent changer.

Pour suivre ces changements automatiquement, les algorithmes doivent adopter un apprentissage **en ligne** ou **incrémental**.

Cela signifie que les données sont traitées progressivement, en un seul passage, sans devoir réentraîner entièrement le modèle depuis le début.

Difficulté de l'apprentissage continu

Il est souvent possible de réaliser la détection en ligne si le modèle entraîné reste fixe.

Mais il est beaucoup plus difficile de développer des algorithmes capables de mettre à jour dynamiquement leur modèle afin de s'adapter à de nouveaux événements ou à de nouveaux motifs.

L'apprentissage continu reste donc un problème ouvert.

Les auteurs classent comme **online** les approches capables de mettre à jour dynamiquement leur modèle, au moins dans une certaine mesure. Les autres approches, qui utilisent une phase d'entraînement hors ligne, sont classées comme **offline**.

Exemples d'adaptation dynamique

Plusieurs stratégies sont mentionnées.

Une première approche consiste à mettre à jour les poids du réseau neuronal lorsque des faux positifs sont identifiés pendant la détection. L'objectif est de corriger la distribution de probabilité des événements sans devoir réentraîner complètement le modèle.

Cependant, cette stratégie suppose souvent une intervention humaine, car quelqu'un doit identifier les faux positifs.

Une autre approche consiste à réentraîner automatiquement le réseau sur de nouveaux lots de logs lorsque le taux de faux positifs dépasse un certain seuil. Mais cette méthode dépend aussi de données étiquetées, ce qui ne supprime pas réellement l'humain de la boucle.

Une solution plus prometteuse est l'utilisation d'un classificateur évolutif, conçu spécialement pour gérer des flux de données instables. Ce type d'approche pourrait mieux prendre en charge l'apprentissage continu.

4. Apprentissage supervisé, semi-supervisé et non supervisé

Les modèles en ligne comme hors ligne peuvent fonctionner selon différents modes d'apprentissage :

- supervisé ;
- semi-supervisé ;
- non supervisé.

L'article observe que presque toutes les approches supervisées utilisent une phase d'entraînement hors ligne.

Cette situation est logique, car l'apprentissage supervisé nécessite des étiquettes. Or ces étiquettes proviennent généralement d'une analyse humaine ou d'une validation réalisée après coup. Elles sont donc plus faciles à produire pour des jeux de données délimités que pour des flux de logs continus.

Les auteurs remarquent aussi que plusieurs approches affirment être non supervisées, alors qu'elles fonctionnent en réalité de manière semi-supervisée.

La raison est simple : elles supposent souvent que les données d'entraînement ne contiennent que des exemples normaux, sans anomalies.

Si des anomalies sont présentes dans les données d'entraînement, elles risquent d'être apprises comme normales par le modèle, ce qui dégrade ensuite la capacité de détection.

Ainsi, seules les architectures réellement conçues pour l'apprentissage non supervisé peuvent supporter la présence d'anomalies dans les données d'entraînement.

IV.C. Préparation des données de logs

Cette section décrit toutes les étapes nécessaires pour préparer des données de logs brutes et textuelles afin qu'elles puissent être utilisées par des réseaux neuronaux. Cela inclut le regroupement des événements en fenêtres ou en sessions, la tokenisation, l'analyse syntaxique des logs, l'extraction de caractéristiques, ainsi que la transformation de ces caractéristiques en représentations vectorielles adaptées aux modèles d'apprentissage profond. La figure 3 de l'article illustre ce pipeline : **données brutes** → **prétraitement** → **extraction** → **représentation numérique**.

1. Prétraitement

Comme les données de logs sont généralement non structurées, il est nécessaire de les prétraiter avant de les introduire dans des systèmes d'apprentissage profond.

La revue montre qu'il existe deux grandes méthodes pour traiter ces données non structurées.

La première méthode, la plus courante, consiste à utiliser des **parseurs**, souvent appelés **log keys**. Ces parseurs permettent d'extraire, pour chaque ligne de log, un identifiant unique d'événement ainsi que les valeurs des paramètres associés.

Les auteurs réutilisent souvent des log keys déjà existantes pour les jeux de données bien connus. Dans d'autres cas, ils créent ces clés à l'aide de méthodes modernes de génération de parseurs, comme **Drain** ou **Spell**.

En général, chaque ligne de log correspond exactement à une clé. Il devient alors facile d'associer un identifiant de type d'événement à chaque ligne pendant l'étape appelée **event mapping**, c'est-à-dire le mappage des événements.

Par exemple, dans la figure 3, la ligne L1 est associée à l'identifiant d'événement E1, car elle correspond à la log key correspondante. La figure montre aussi que l'analyse syntaxique permet d'extraire plusieurs paramètres : l'horodatage, l'identifiant du PacketResponder et l'identifiant du bloc de fichier.

La deuxième méthode repose sur des stratégies fondées sur les **tokens**. Elle consiste à découper les messages de logs en listes de mots, par exemple en les séparant au niveau des espaces.

Il est ensuite courant de nettoyer les données :

- en mettant les lettres en minuscules ;
- en supprimant certains caractères spéciaux ;
- en retirant les mots peu informatifs.

Ces approches extraient moins d'information sémantique que les parseurs, mais elles sont plus flexibles. Elles reposent sur des heuristiques générales et non sur des parseurs prédéfinis. Elles peuvent donc être appliquées plus facilement à différents types de logs.

Certaines approches combinent les deux stratégies : elles utilisent à la fois le parsing et la tokenisation. Par exemple, elles peuvent générer des vecteurs de tokens à partir d'événements déjà analysés plutôt qu'à partir des lignes brutes.

2. Regroupement des événements

Comme expliqué dans la partie théorique, la détection d'anomalies ponctuelles simples ne nécessite pas forcément de regrouper les logs. Un événement isolé inhabituel peut être considéré comme anormal indépendamment du contexte.

Cependant, l'apprentissage profond est très souvent utilisé pour détecter des motifs inhabituels portant sur plusieurs événements, par exemple :

- des changements dans les séquences d'événements ;
- des corrélations temporelles inhabituelles ;
- des changements de fréquence ;
- des motifs collectifs anormaux.

Dans ces cas, il devient nécessaire d'organiser logiquement les événements en groupes, puis d'analyser ces groupes individuellement ou les uns par rapport aux autres.

La figure 3 présente plusieurs stratégies courantes de regroupement.

Fenêtres temporelles

Le regroupement par fenêtres temporelles est presque toujours possible, car les événements de logs contiennent généralement un horodatage.

Comme l'horodatage apparaît souvent au début des lignes de logs, il n'est pas nécessaire d'analyser toute la ligne pour effectuer ce regroupement.

Il existe deux grandes stratégies de regroupement temporel.

La première est celle des **fenêtres glissantes**. Une fenêtre glissante possède une durée fixe et se déplace dans les données avec un pas donné. Ce pas est généralement inférieur à la taille de la fenêtre. À chaque déplacement, toutes les lignes de logs dont l'horodatage se trouve entre le début et la fin de la fenêtre sont regroupées ensemble.

L'avantage est que cette méthode offre une vue fine de l'évolution temporelle. L'inconvénient est qu'une même ligne de log peut appartenir à plusieurs fenêtres, puisque les fenêtres se chevauchent.

La deuxième stratégie est celle des **fenêtres fixes**. Il s'agit d'un cas particulier des fenêtres glissantes, où le pas de déplacement est égal à la taille de la fenêtre. Dans ce cas, chaque événement appartient à une seule fenêtre.

Cette méthode est moins fine, mais elle facilite certains calculs, notamment l'analyse de séries temporelles, car chaque événement est compté une seule fois.

Les auteurs précisent qu'un regroupement peut aussi être effectué non pas selon le temps, mais selon un nombre fixe de lignes. Cela permet d'avoir des groupes de même taille, mais rend plus difficile l'interprétation des fréquences d'événements en fonction du temps.

Fenêtres de session

Une autre stratégie consiste à utiliser des **fenêtres de session**.

Cette méthode repose sur un paramètre d'événement qui joue le rôle d'identifiant pour une tâche ou un processus particulier. En regroupant les événements selon cet identifiant, on peut reconstituer des séquences correspondant mieux aux workflows réels du programme.

Cette stratégie est utile lorsque plusieurs processus s'exécutent simultanément. Même si les logs sont mélangés dans le fichier global, l'identifiant de session permet de retrouver les événements appartenant à une même activité.

La limite est que tous les types de logs ne contiennent pas un tel identifiant de session.

Dans l'exemple de la figure 3, l'identifiant de bloc de fichier, par exemple blk_388, joue ce rôle. Il permet d'extraire plusieurs séquences d'événements correspondant à différents blocs.

3. Extraction de caractéristiques

Les stratégies de prétraitement fondées sur le parsing ou sur les tokens permettent d'extraire des caractéristiques structurées à partir de logs qui étaient initialement non structurés.

Certaines caractéristiques sont directement issues du prétraitement.

Par exemple, une **séquence de tokens** peut être utilisée sans modification importante pour analyser chaque ligne de log comme une phrase composée de mots.

À l'inverse, les **comptages de tokens** nécessitent une étape supplémentaire : il faut comparer et compter les tokens présents dans chaque ligne. Des techniques de pondération peuvent aussi être utilisées, comme **TF-IDF**, qui estime l'importance d'un token en fonction de sa fréquence dans une ligne et de sa rareté dans l'ensemble des lignes observées.

Après l'étape de mappage des événements, il devient simple d'extraire des **séquences d'événements**. Ces séquences sont des suites d'identifiants de types d'événements, généralement séparées selon des fenêtres fixes, glissantes ou de session.

Dans l'exemple de la figure 3, la séquence d'événements du bloc blk_388 est :

[E1, E1, E2, E1]

Cela correspond aux identifiants des log keys associées aux lignes L1 à L4.

Les **comptages d'événements** sont représentés sous forme de vecteurs. Si l'on dispose de d log keys, le vecteur contient d éléments. Le i -ème élément indique combien de fois la i -ème log key apparaît dans le groupe étudié.

Par exemple, le vecteur :

[3, 1, 0, 0]

signifie que le premier type d'événement apparaît trois fois, le deuxième une fois, et les deux autres pas du tout.

D'autres propriétés statistiques peuvent être calculées à partir des occurrences d'événements, par exemple :

- le pourcentage de logs saisonniers ;
- la longueur des messages de logs ;
- le taux d'activité des logs ;
- des scores fondés sur l'entropie ;
- la présence de pics soudains dans les occurrences d'événements.

Les **paramètres** des événements de logs sont extraits sous forme de listes de valeurs. Comme leur signification sémantique est connue après le parsing, chaque valeur peut être analysée avec une méthode adaptée à son type : numérique, catégoriel, temporel, etc.

Un paramètre particulièrement important est l'horodatage, car il permet de placer les événements dans un ordre chronologique et d'étudier leurs dépendances dynamiques.

Les **temps d'intervalle entre événements**, c'est-à-dire les durées entre deux occurrences d'un même type d'événement, constituent donc une caractéristique fréquemment utilisée pour la détection d'anomalies.

4. Représentation des caractéristiques

Les caractéristiques extraites sont ensuite transformées en vecteurs numériques ou catégoriels utilisables par des réseaux neuronaux.

Par exemple, les séquences d'événements peuvent être représentées sous forme de **vecteurs de séquences d'identifiants d'événements**. Ces séquences ordonnées peuvent être données à des RNN afin que le modèle apprenne les dépendances entre événements et détecte des motifs séquentiels inhabituels.

Les comptages d'événements peuvent être représentés sous forme de **vecteurs de comptage**. Les statistiques d'événements peuvent être représentées sous forme de **vecteurs de caractéristiques statistiques**, où chaque position du vecteur correspond à une propriété particulière.

Cependant, beaucoup d'approches ne se contentent pas d'utiliser directement ces caractéristiques. Elles les transforment ou les combinent pour obtenir des représentations plus riches.

Vecteurs sémantiques

La représentation la plus fréquente est le **vecteur sémantique**.

Dans le traitement automatique du langage naturel, il est courant de transformer les mots d'une phrase en vecteurs capables d'encoder leur contexte ou certaines propriétés linguistiques. Des méthodes comme **Word2Vec**, **BERT**, **GloVe** ou **TF-IDF** sont utilisées pour cela.

Comme une ligne de log est souvent une séquence de tokens comparable à une phrase, il est logique d'appliquer des méthodes issues du traitement du langage naturel aux logs.

De la même manière, une séquence d'événements peut être vue comme une phrase, où chaque log key représente un mot.

L'encodage sémantique est généralement obtenu soit en entraînant un modèle neuronal sur un fichier de logs spécifique, soit en utilisant des modèles déjà préentraînés.

Encodage positionnel

Les vecteurs sémantiques sont parfois combinés avec un **encodage positionnel**.

L'objectif est d'ajouter l'information de position des éléments dans une séquence. En effet, dans une séquence de logs, l'ordre des événements est souvent important.

Pour ajouter cette information, certains auteurs utilisent des fonctions sinus et cosinus selon les indices pairs ou impairs des tokens.

One-hot encoding

Le **one-hot encoding** est l'une des techniques les plus courantes pour représenter des données catégorielles.

Elle est fréquemment appliquée aux types d'événements, parce qu'un nombre fini de log keys est défini par le parseur.

Si une valeur i appartient à une liste ordonnée de d valeurs, son encodage one-hot est un vecteur de longueur d , dans lequel le i -ème élément vaut 1 et tous les autres valent 0.

La plupart des auteurs utilisent directement ces vecteurs comme entrée du réseau neuronal. D'autres les combinent avec des vecteurs de comptage pour permettre au modèle d'apprendre des comportements différents selon les types de logs.

Couches ou matrices d'embedding

Les **couches d'embedding** ou **matrices d'embedding** sont généralement utilisées pour résoudre les problèmes de sparsité des données de grande dimension.

Ce problème apparaît notamment lorsque l'on utilise des vecteurs one-hot pour représenter un grand nombre de types d'événements.

Ces matrices sont généralement initialisées aléatoirement, puis entraînées en même temps que le modèle de classification. Leur but est de produire de meilleurs encodages vectoriels des messages de logs.

La différence avec les vecteurs sémantiques est importante : les embeddings ne sont pas nécessairement entraînés avec un objectif linguistique comme Word2Vec. Ils sont plutôt optimisés pour minimiser la fonction de perte du modèle de classification.

Deep encoded embeddings

Certains auteurs utilisent des modèles d'embedding plus complexes fondés sur l'apprentissage profond. Les auteurs appellent ces représentations des **deep encoded embeddings**.

Elles peuvent combiner plusieurs types d'informations :

- des embeddings fondés sur les caractères ;
- des embeddings fondés sur les événements ;
- des embeddings fondés sur les séquences ;
- des mécanismes d'attention ;
- des CNN ;
- des VAE.

L'objectif est d'obtenir une représentation plus riche que les simples identifiants ou les vecteurs one-hot.

Vecteurs de paramètres

Certaines approches utilisent directement les valeurs des paramètres extraits des messages de logs.

Les paramètres numériques peuvent être utilisés pour des analyses de séries temporelles multivariées avec des RNN. Les paramètres catégoriels peuvent être encodés en one-hot ou transformés par des méthodes d'embedding.

Cependant, les événements de logs n'ont pas toujours le même nombre de paramètres ni la même signification sémantique. Il est donc souvent nécessaire d'analyser séparément les paramètres de chaque type d'événement.

L'horodatage constitue encore une fois un paramètre particulier. Il permet d'interpréter les autres valeurs dans un contexte temporel. Dans certains travaux, les horodatages eux-mêmes sont transformés en représentations vectorielles par **time embedding**.

Graphes et matrices de transition

Une approche plus rare consiste à représenter les logs sous forme de **graphes**.

Dans ce cas, les séquences d'événements sont transformées en graphes de session :

- les sommets représentent les événements ;
- les arêtes représentent les transitions entre événements.

Ces graphes peuvent ensuite être traités par des Graph Neural Networks.

Une autre stratégie consiste à utiliser une **matrice de transition**. Cette matrice encode les probabilités qu'un type d'événement suive un autre type d'événement.

Ainsi, au lieu de représenter seulement une séquence, on représente explicitement les probabilités de passage entre événements.

IV.D. Techniques de détection d'anomalies

Les sections précédentes ont décrit comment préparer les données de logs afin qu'elles puissent être utilisées par des réseaux neuronaux. Cependant, dans beaucoup de cas, il n'est pas évident de savoir si un échantillon présenté au réseau est normal ou anormal. La difficulté vient du fait que les anomalies ne sont pas toujours connues à l'avance et qu'il n'existe donc pas forcément de neurones de sortie explicitement dédiés à chaque type d'anomalie.

Cette section présente donc les techniques utilisées pour transformer la sortie du réseau neuronal en décision de détection.

Les auteurs procèdent en trois étapes :

1. identifier les types d'anomalies généralement ciblés ;
 2. expliquer comment la sortie du réseau neuronal est exploitée ;
 3. montrer comment les échantillons normaux et anormaux sont différenciés.
-

1. Types d'anomalies

Les approches étudiées détectent plusieurs types d'anomalies.

a. Anomalies ponctuelles ou outliers

Les **outliers** sont des événements de logs isolés qui ne correspondent pas à la structure générale du jeu de données.

Ils peuvent être détectés à partir de plusieurs éléments inhabituels :

- des valeurs de paramètres anormales ;
- des séquences de tokens inhabituelles ;
- des moments d'occurrence anormaux.

Cependant, relativement peu d'approches se concentrent uniquement sur ces anomalies ponctuelles. La majorité des travaux étudiés s'intéresse plutôt aux anomalies collectives, notamment celles qui impliquent des séquences d'événements.

b. Anomalies séquentielles

Les **anomalies séquentielles** apparaissent lorsque les chemins d'exécution changent.

Autrement dit, les applications qui produisent les logs exécutent les événements dans un ordre différent de celui qui est habituellement observé.

Cela peut se manifester par :

- des événements supplémentaires ;
- des événements manquants ;
- des événements placés dans un ordre inhabituel ;
- des séquences entièrement nouvelles ;
- l'apparition de types d'événements jamais vus auparavant.

Une méthode courante consiste à vérifier si un certain type d'événement est attendu dans une séquence donnée, en tenant compte des événements qui le précèdent ou qui le suivent.

Par exemple, si le système apprend que la séquence normale est :

Connexion → Authentification → Accès ressource → Déconnexion

alors une séquence comme :

Connexion → Accès ressource → Authentification → Déconnexion

peut être considérée comme suspecte, car l'ordre logique des événements est modifié.

c. Anomalies de fréquence

Les **anomalies de fréquence** ne s'intéressent pas principalement à l'ordre des événements, mais au nombre d'occurrences de certains événements.

L'idée est que certains comportements normaux produisent des fréquences relativement stables.

Par exemple, dans un système de fichiers, le nombre d'événements liés à l'ouverture de fichiers et le nombre d'événements liés à la fermeture de fichiers devraient être cohérents. Un fichier doit normalement être ouvert avant d'être fermé, et chaque fichier ouvert doit finir par être fermé.

Si ces fréquences deviennent incohérentes, cela peut signaler une anomalie.

La détection des anomalies de fréquence repose donc souvent sur le comptage des événements dans des fenêtres temporelles.

d. Anomalies statistiques

Certaines approches vont au-delà du simple comptage d'événements.

Elles analysent des propriétés quantitatives plus générales, par exemple :

- les temps d'arrivée entre deux événements ;
- les motifs saisonniers d'apparition des événements ;
- les distributions temporelles ;
- les variations statistiques dans les séries de logs.

Les auteurs appellent ces anomalies des **anomalies statistiques**, car leur détection suppose généralement que les événements suivent certaines distributions relativement stables dans le temps.

2. Sortie du réseau neuronal

De manière générale, la sortie d'un réseau neuronal est composée soit d'un seul neurone, soit de plusieurs neurones dans la couche finale.

La valeur produite peut donc être :

- un scalaire ;
- un vecteur numérique ;
- une distribution de probabilités ;
- une étiquette de classe ;
- une erreur de reconstruction ;
- un score d'anomalie.

Une première possibilité consiste à interpréter cette sortie comme un **score d'anomalie**. Ce score indique dans quelle mesure les logs présentés au réseau semblent normaux ou anormaux.

Cependant, un score d'anomalie est rarement interprétable seul. Il faut généralement le comparer à un **seuil**.

Si le score dépasse ce seuil, l'échantillon est déclaré anormal. Sinon, il est considéré comme normal.

a. Classification binaire

Dans la **classification binaire**, le réseau neuronal estime directement si l'entrée est :

normale

ou

anormale

Cette approche est simple à exploiter, mais elle suppose généralement que l'on dispose de données étiquetées. Elle est donc surtout compatible avec l'apprentissage supervisé.

b. Transformations de vecteurs d'entrée

Certaines approches transforment les données d'entrée dans un nouvel espace vectoriel.

Dans cet espace, les données normales doivent former des groupes ou clusters cohérents. Les anomalies sont alors détectées parce qu'elles se trouvent loin des centres de ces groupes.

Cette logique est proche de certaines méthodes de clustering ou de représentation latente.

c. Erreur de reconstruction

Les autoencodeurs utilisent une stratégie différente.

Ils encodent d'abord les données d'entrée dans un espace de plus faible dimension, puis essaient de reconstruire les données originales.

Si l'échantillon ressemble aux données normales apprises pendant l'entraînement, il sera bien reconstruit.

En revanche, si l'échantillon est inhabituel, l'autoencodeur aura plus de difficulté à le reconstruire. L'erreur de reconstruction sera alors élevée.

Dans ce cas, l'échantillon est considéré comme anormal.

d. Classification multi-classe

La **classification multi-classe** permet de distinguer plus de deux classes.

Elle peut, par exemple, attribuer des étiquettes différentes à plusieurs types d'anomalies.

Mais cette méthode nécessite généralement un apprentissage supervisé, car le modèle doit apprendre les motifs propres à chaque classe pendant l'entraînement.

Cette contrainte limite son utilisation dans la détection d'anomalies, car les anomalies connues et bien étiquetées sont souvent rares.

e. Prédiction du prochain événement

Pour contourner le manque d'étiquettes d'anomalies, certaines approches ne cherchent pas directement à prédire si une séquence est normale ou anormale.

Elles entraînent plutôt le modèle à prédire le prochain type d'événement attendu dans une séquence de logs.

Lorsque la couche de sortie utilise une fonction **softmax**, le modèle produit une distribution de probabilités sur les différentes log keys possibles.

L'anomalie est alors détectée indirectement : si l'événement réellement observé ne fait pas partie des événements les plus probables selon le modèle, la séquence est considérée comme suspecte.

f. Régression sur vecteurs numériques

Le problème de prédiction du prochain événement peut aussi être formulé comme une tâche de régression.

Dans ce cas, les événements ne sont pas considérés comme de simples classes, mais comme des vecteurs numériques, par exemple :

- des vecteurs sémantiques ;
- des vecteurs de paramètres ;
- des embeddings.

Le réseau produit alors un vecteur représentant l'événement attendu. On compare ensuite ce vecteur prédit avec le vecteur réellement observé.

Un écart important peut indiquer une anomalie.

3. Méthodes de décision

Les différentes sorties du réseau neuronal donnent lieu à plusieurs méthodes pour décider si un comportement est normal ou anormal.

a. Décision par étiquette

Lorsque la sortie du réseau correspond directement à une étiquette, la décision est simple.

Si l'échantillon reçoit l'étiquette :

anormal

alors il est signalé comme anomalie.

Cette méthode est surtout utilisée dans les approches de classification binaire ou multi-classe.

b. Décision par seuil

Lorsque le réseau produit un score numérique ou un score d'anomalie, il faut définir un seuil.

Si le score est supérieur au seuil, l'échantillon est considéré comme anormal.

Ce seuil permet d'ajuster les performances du système. Il sert notamment à trouver un compromis entre :

- le taux de vrais positifs ;
- le taux de faux positifs.

Un seuil trop bas augmente la sensibilité du modèle, mais peut produire beaucoup de fausses alertes.

Un seuil trop haut réduit les fausses alertes, mais peut laisser passer certaines anomalies.

c. Décision par distribution statistique

Une autre stratégie consiste à modéliser les scores d'anomalie comme des distributions statistiques.

Par exemple, certaines approches utilisent une distribution gaussienne pour détecter les vecteurs de paramètres dont l'erreur se situe en dehors d'un intervalle de confiance donné.

Dans ce cas, l'anomalie n'est pas seulement définie par un seuil fixe, mais par un écart statistiquement significatif par rapport au comportement attendu.

d. Décision par clustering

Certaines approches appliquent du clustering sur les erreurs de reconstruction.

Les échantillons proches des clusters normaux sont considérés comme normaux.

Les échantillons suffisamment éloignés de ces clusters sont considérés comme anormaux.

Cette approche est utile lorsque l'on veut détecter des comportements qui ne correspondent à aucun groupe normal connu.

e. Décision par top-n événements probables

Lorsque le réseau produit une distribution de probabilités sur les log keys connues, une méthode courante consiste à considérer les **n événements les plus probables** comme des candidats acceptables.

Une anomalie est détectée uniquement si le type réel de l'événement observé ne fait pas partie de ces candidats.

Par exemple, si le modèle prédit que les trois événements les plus probables sont :

E1, E4, E7

mais que l'événement réel observé est :

E9

alors la séquence peut être signalée comme anormale.

Le paramètre n joue un rôle semblable à celui d'un seuil.

- Si n est petit, le système devient plus strict.
- Si n est grand, le système devient plus tolérant.

Cela influence directement le compromis entre vrais positifs et faux positifs.

4. Lecture de la figure 4

La figure 4 de l'article résume les combinaisons les plus fréquentes entre les types de sorties du réseau neuronal et les méthodes de détection.

Elle montre notamment que :

- la classification binaire et la classification multi-classe reposent généralement sur l'apprentissage supervisé ;
- ces méthodes attribuent directement des étiquettes aux nouveaux échantillons ;
- les autres méthodes permettent plutôt un apprentissage semi-supervisé ou non supervisé ;
- les erreurs de reconstruction, les transformations vectorielles et les vecteurs numériques produisent des scores comparés à des seuils ;
- les distributions de probabilités sont généralement comparées aux log keys les plus probables.

Les auteurs signalent toutefois qu'il existe des exceptions. Certaines approches semi-supervisées utilisent des étiquettes probabilistes, tandis que certaines approches supervisées reposent malgré tout sur des erreurs de reconstruction.

IV.E. Évaluation et reproductibilité

Cette section présente une vue d'ensemble des évaluations réalisées dans les publications étudiées. Les auteurs y recensent les jeux de données disponibles publiquement, les métriques d'évaluation couramment utilisées, les approches de référence servant de comparaison, ainsi que les problèmes de reproductibilité liés à la disponibilité du code source.

1. Jeux de données

Les jeux de données sont essentiels dans les publications scientifiques, car ils permettent de valider les approches proposées et de montrer leurs améliorations par rapport à l'état de l'art.

La revue montre cependant qu'il existe seulement quelques jeux de données couramment utilisés pour évaluer les approches de détection d'anomalies dans les logs à l'aide de l'apprentissage profond.

La grande majorité des évaluations repose principalement sur quatre jeux de données :

- **HDFS** ;
- **BGL** ;
- **Thunderbird** ;
- **OpenStack**.

Le tableau IV de l'article présente les jeux de données publics les plus utilisés, avec leur année, leur cas d'usage, la présence ou non d'étiquettes, la présence ou non d'identifiants de session, et la source des anomalies.

a. HDFS

Le jeu de données **HDFS** provient du système de fichiers distribué Hadoop, exécuté sur un cluster de calcul haute performance composé de **203 nœuds**.

Plus de **24 millions de logs** ont été collectés sur une période de deux jours. Ce jeu de données contient des séquences d'événements hétérogènes liées à des blocs de fichiers spécifiques. Ces blocs servent d'identifiants de session.

Certaines séquences correspondent à des chemins d'exécution anormaux, principalement liés à des problèmes de performance, comme des exceptions d'écriture. Ces anomalies ont été étiquetées manuellement.

Les exemples de logs présentés dans la figure 3 de l'article sont des versions simplifiées de logs issus du jeu de données HDFS.

b. BGL, Thunderbird, Spirit et HPC RAS

Le jeu de données **BGL** contient plus de **quatre millions d'événements de logs** collectés pendant plus de **200 jours** sur un supercalculateur BlueGene/L du Lawrence Livermore National Laboratory.

Les événements possèdent un champ de sévérité permettant de les séparer en classes. Cependant, les logs ont aussi été étiquetés manuellement par des administrateurs système.

Les anomalies présentes dans BGL correspondent à des problèmes matériels et logiciels.

Les jeux de données **Thunderbird** et **Spirit** appartiennent à la même famille. Ils contiennent des logs système de type syslog et incluent également des alertes liées à des problèmes système, par exemple des pannes de disque.

Les événements de Thunderbird et Spirit contiennent des balises d'alerte générées automatiquement, qui peuvent servir d'étiquettes.

Le jeu de données **HPC RAS** appartient également à cette famille de logs de supercalculateurs. Les logs RAS, c'est-à-dire Reliability, Availability and

Serviceability, servent généralement à comprendre les causes des défaillances système. Toutefois, ce jeu de données ne contient pas d'étiquettes d'anomalies.

c. OpenStack

Le jeu de données **OpenStack** a été collecté à partir d'une plateforme OpenStack où des scripts automatiques exécutent continuellement et aléatoirement des tâches liées à la gestion de machines virtuelles.

Ces tâches incluent notamment :

- la création de machines virtuelles ;
- la mise en pause ;
- la suppression ;
- d'autres opérations de gestion.

Contrairement aux jeux de données précédents, qui contiennent surtout des défaillances survenues de manière plus ou moins naturelle, les auteurs du jeu de données OpenStack ont volontairement injecté des anomalies à des moments précis.

Ces anomalies incluent, par exemple, des délais d'expiration et des erreurs.

Les événements contiennent aussi des identifiants d'instance, qui peuvent être utilisés pour identifier des sessions.

d. Hadoop

Le jeu de données **Hadoop** ressemble au jeu HDFS, car il provient aussi d'un cluster de calcul exécutant des tâches MapReduce.

Les tâches utilisées incluent notamment :

- **WordCount** ;
- **PageRank**.

Après une phase initiale d'entraînement, les auteurs déclenchent volontairement des défaillances sur les nœuds. Ces défaillances incluent :

- l'arrêt d'une machine ;
- la déconnexion du réseau ;
- le remplissage du disque dur d'un serveur.

Les logs sont séparés en différents fichiers selon des identifiants d'application. Ces identifiants fonctionnent de manière similaire à des identifiants de session et sont utilisés pour attribuer des étiquettes aux exécutions anormales.

e. Loghub, Spark, ZooKeeper, Windows, Android et Linux

Loghub regroupe plusieurs jeux de données utilisés dans les évaluations.

Le jeu de données **Spark** contient des logs provenant d'Apache Spark, un moteur distribué de traitement de données exécuté sur **32 machines**. Ces logs incluent des exécutions normales et anormales, mais ils ne sont pas étiquetés.

ZooKeeper est un autre service Apache utilisé dans le calcul distribué et la gestion de configuration.

Loghub contient aussi des logs issus de systèmes d'exploitation classiques.

Le jeu de données **Windows** a été collecté sur une machine Windows 7 en laboratoire, en surveillant le composant CBS, c'est-à-dire Component Based Servicing, qui opère au niveau des paquets et des mises à jour.

Le jeu de données **Android** a été collecté sur un téléphone mobile dans un environnement de laboratoire.

Ces deux jeux de données, Windows et Android, ne sont pas étiquetés et ne contiennent pas d'anomalies volontairement injectées.

Des logs Linux sont également fournis dans les images disque du projet Digital Corpora. Ces logs contiennent certains problèmes, comme des authentifications invalides.

f. Jeux de données orientés sécurité

Les jeux de données précédents contiennent surtout des événements de défaillance générés dans le cadre d'une utilisation légitime du système.

D'autres jeux de données contiennent plutôt des événements produits par des activités malveillantes. Ils permettent donc d'évaluer des systèmes de détection d'intrusion fondés sur les anomalies.

Le jeu de données **DFRWS 2009** contient des logs système provenant de dispositifs Linux et impliquant :

- de l'exfiltration de données ;
- des accès non autorisés ;
- l'utilisation de logiciels de porte dérobée.

Les jeux de données **Honeynet 2010** et **Honeynet 2011** contiennent des fichiers de logs communs provenant de machines Linux compromises, ayant subi des accès illégitimes.

Le site **Public Security Log Sharing Site** fournit également des Linux Security Logs issus de différentes sources et affectés par diverses intrusions réelles, comme les attaques par force brute.

Le problème est que ces fichiers de logs de sécurité ne contiennent généralement pas d'étiquettes précises pour les événements malveillants. Les chercheurs doivent donc créer leurs propres vérités terrain pour évaluer leurs approches sur ces jeux de données.

2. Métriques d'évaluation

L'évaluation quantitative des approches de détection d'anomalies repose généralement sur le comptage de quatre types de résultats.

Un **vrai positif** correspond à un échantillon anormal correctement détecté.

Un **faux positif** correspond à un échantillon normal détecté à tort comme anormal.

Un **faux négatif** correspond à une anomalie que le système n'a pas détectée.

Un **vrai négatif** correspond à un échantillon normal correctement considéré comme normal.

Lorsque les événements sont étiquetés individuellement et que les échantillons correspondent à des événements isolés, comme dans BGL, l'évaluation par classification binaire est relativement directe.

Cependant, certains travaux considèrent aussi des problèmes de classification multi-classe lorsque différents types de pannes disposent d'étiquettes distinctes. Dans ce cas, les auteurs calculent parfois des moyennes de métriques sur l'ensemble des classes ou produisent des matrices de confusion.

Difficulté liée à l'agrégation des logs

L'évaluation devient moins directe lorsque les logs sont regroupés avant la détection.

Par exemple, si les événements sont agrégés dans des fenêtres temporelles, il peut être nécessaire de considérer une détection comme correcte lorsqu'elle est suffisamment proche de l'anomalie réelle dans la séquence.

De plus, beaucoup d'articles utilisent HDFS, où les étiquettes ne sont pas disponibles pour chaque événement individuel, mais plutôt pour des sessions entières. Dans ce cas, la méthode la plus courante consiste à compter les sessions normales ou anormales correctement ou incorrectement identifiées.

Cela montre une limite importante : deux articles peuvent utiliser les mêmes métriques, mais ne pas compter les vrais positifs, faux positifs, faux négatifs et vrais négatifs exactement de la même manière.

Métriques courantes

Les métriques les plus utilisées sont :

La précision, qui mesure la proportion des alertes détectées comme anomalies qui sont réellement des anomalies.

Le rappel, aussi appelé taux de vrais positifs, qui mesure la proportion des anomalies réelles effectivement détectées.

Le taux de faux positifs, qui mesure la proportion d'échantillons normaux signalés à tort comme anomalies.

Le score F1, qui combine précision et rappel dans une seule mesure.

Les formules sont les suivantes :

$$\text{Précision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Rappel} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{FPR} = \text{FP} / (\text{FP} + \text{TN})$$

$$\text{F1} = 2 \times (\text{Précision} \times \text{Rappel}) / (\text{Précision} + \text{Rappel})$$

Des métriques moins fréquentes sont aussi utilisées, notamment :

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

L'accuracy apparaît dans **15 publications**.

Certains travaux utilisent aussi l'aire sous la courbe, calculée soit pour des courbes précision-rappel, soit pour des courbes ROC.

D'autres métriques sont plus spécifiques aux applications d'apprentissage profond, par exemple :

- le nombre de paramètres du modèle ;
- le temps nécessaire à l'entraînement ;
- le temps nécessaire à la détection.

Certains auteurs évaluent aussi des aspects plus avancés, comme l'effet de l'entraînement sur plusieurs jeux de données ou la robustesse face aux changements de motifs de logs dans le temps.

3. Approches de référence utilisées comme benchmarks

La plupart des publications comparent leurs résultats avec des approches de référence afin de montrer une amélioration par rapport à l'état de l'art.

La figure 5 de l'article présente les benchmarks les plus utilisés. Les auteurs ne retiennent dans cette figure que les approches apparaissant dans au moins deux publications.

Le benchmark le plus utilisé est **DeepLog**, qui apparaît dans **38 publications sur 62**.

DeepLog repose sur des LSTM pour prédire les prochains événements dans les séquences de logs. Un événement observé est considéré comme anomalie s'il est associé à une faible probabilité selon le modèle.

Sa popularité comme méthode de comparaison s'explique par deux raisons principales :

- DeepLog est l'une des premières approches à détecter des anomalies séquentielles dans les logs avec l'apprentissage profond ;
- des réimplémentations open source sont disponibles.

Le deuxième benchmark le plus utilisé repose sur l'**analyse en composantes principales**, ou **PCA**.

Contrairement à DeepLog, cette approche s'appuie sur des comptages d'événements plutôt que sur des séquences. Elle transforme les vecteurs de comptage en sous-

espaces où les anomalies apparaissent comme des points éloignés des autres échantillons.

Une autre méthode classique est **Invariant Mining**. Elle cherche à découvrir des relations linéaires entre événements de logs représentant des workflows d'exécution. Les séquences qui violent ces invariants sont considérées comme anormales.

LogCluster génère des vecteurs pour les séquences de logs, puis les regroupe par clustering selon une métrique de similarité. Cette approche est principalement conçue pour le filtrage de logs, mais elle peut aussi être utilisée pour détecter des motifs inhabituels.

Les **SVM** utilisent également des vecteurs de comptage d'événements. Comme les SVM sont généralement supervisés, certains auteurs utilisent des variantes adaptées aux contextes semi-supervisés ou non supervisés, comme le one-class SVM ou le Support Vector Data Description.

D'autres benchmarks sont aussi mentionnés :

- **LogAnomaly**, qui utilise des LSTM et une méthode d'embedding appelée template2Vec ;
- **LogRobust**, conçu pour gérer les événements inconnus qui apparaissent lors de l'évolution logicielle ;
- **Isolation Forest**, qui isole récursivement les instances pour détecter les anomalies ;
- **Logistic Regression**, adaptée aux données numériques ;
- **Decision Tree**, qui classe les instances par divisions successives ;
- **CNN**, notamment l'approche de Lu et al. ;
- **nLSAlog**, fondée sur des LSTM.

Sur l'ensemble des publications étudiées, seules **huit** ne comparent pas leur approche à un benchmark.

4. Reproductibilité

La reproductibilité est essentielle dans les publications scientifiques.

Elle permet à d'autres chercheurs :

- de vérifier la validité des résultats ;
- d'étendre les algorithmes proposés ;
- de tester les approches sur d'autres jeux de données ;
- d'utiliser les approches comme benchmarks dans de nouvelles publications.

Les auteurs considèrent qu'une approche est reproductible lorsque deux conditions sont réunies :

1. les données utilisées pour l'évaluation sont accessibles publiquement ;
2. le code source original est disponible publiquement.

La majorité des publications évaluent leurs approches sur quelques jeux de données publics. Certains travaux utilisent aussi des jeux de données privés :

- générés synthétiquement dans des bancs d'essai ;
- collectés dans des institutions académiques ;
- obtenus à partir d'applications industrielles réelles.

Au total, **55 publications sur 62** utilisent au moins un jeu de données public présenté dans le tableau IV.

Cependant, la disponibilité du code source reste beaucoup plus faible. Les auteurs n'ont trouvé le code source original que pour **8 publications**. Ils signalent aussi l'existence de quelques réimplémentations dans la boîte à outils **deep-loglizer**.

Cela signifie que, même si les jeux de données sont souvent accessibles, les expériences restent difficiles à reproduire exactement lorsque le code original n'est pas publié.

V. Discussion

Les sections précédentes ont présenté les résultats détaillés de la revue. Dans cette partie, les auteurs synthétisent ces résultats, discutent les problèmes ouverts de la détection d'anomalies dans les logs avec l'apprentissage profond, puis répondent directement aux sept questions de recherche formulées au début de l'article.

RQ1 — Quels sont les principaux défis de la détection d'anomalies dans les logs avec l'apprentissage profond ?

Les auteurs constatent que l'un des défis les plus importants est **l'instabilité des données**. Dans les systèmes réels, de nouveaux événements peuvent apparaître après une mise à jour logicielle, un changement d'architecture ou une modification de l'usage du système. Les modèles doivent donc pouvoir gérer des événements inconnus, sinon ils risquent de les considérer à tort comme anormaux.

Un autre défi majeur est la **représentation des données**. Les logs sont souvent textuels, hétérogènes et semi-structurés ou non structurés. Or les réseaux neuronaux nécessitent généralement des données numériques. Il faut donc transformer les logs en vecteurs, embeddings, matrices, graphes ou représentations statistiques.

Le troisième défi concerne le **traitement incrémental et en flux**. Les logs sont générés continuellement. Pour une détection réellement opérationnelle, il ne suffit pas d'entraîner un modèle hors ligne : il faut aussi pouvoir analyser les événements au fur et à mesure de leur arrivée et adapter le modèle lorsque le comportement normal du système évolue.

Les auteurs soulignent également que la diversité des anomalies reste insuffisamment couverte. La majorité des travaux se concentre sur les séquences et les fréquences d'événements, alors que d'autres formes d'anomalies peuvent apparaître dans les paramètres, les corrélations, les distributions statistiques, les comportements périodiques ou les temps entre événements.

Enfin, l'**explicabilité** reste un problème sérieux. Les modèles profonds peuvent produire une alerte sans expliquer clairement pourquoi un événement ou une séquence a été considéré comme anormal. Cela limite leur utilité pour l'analyse de cause racine et complique le travail des opérateurs système.

RQ2 — Quels algorithmes d'apprentissage profond sont généralement utilisés ?

La revue montre que plusieurs familles de modèles sont utilisées, mais les **RNN** dominent largement. Cette domination s'explique par la nature séquentielle des logs : beaucoup d'anomalies se manifestent par des changements dans l'ordre des événements.

Les **CNN** sont aussi utilisés comme alternative efficace, car ils peuvent capturer certaines dépendances entre événements tout en étant parfois moins coûteux que les RNN.

Les **autoencodeurs** et les **Transformers** sont également fréquents, notamment parce qu'ils peuvent soutenir des approches semi-supervisées ou non supervisées.

Les **GAN**, **MLP**, **GNN** et **EGNN** sont moins fréquents, mais les auteurs considèrent qu'ils restent intéressants. Leur faible utilisation ne signifie pas qu'ils sont inutiles ; cela montre plutôt que la recherche s'est concentrée sur un petit nombre de directions, surtout les séquences d'événements.

Les auteurs suggèrent aussi que d'autres architectures pourraient être explorées, par exemple les **Deep Belief Networks** ou l'**apprentissage par renforcement profond**, surtout pour des scénarios où les anomalies ne sont pas seulement séquentielles.

RQ3 — Comment les logs sont-ils prétraités pour être utilisés par des modèles d'apprentissage profond ?

Les auteurs identifient trois grandes stratégies.

La première consiste à **tokeniser les messages de logs**. On découpe les messages en mots ou tokens. Cette méthode est simple et ne nécessite pas de parseur spécialisé, mais elle peut perdre une partie du sens réel des événements, car elle traite les logs comme du texte ordinaire.

La deuxième consiste à **parser les logs** afin d'extraire des informations structurées. On identifie alors des modèles d'événements, appelés log keys ou templates, puis on extrait des séquences, des comptages ou des statistiques à partir de ces événements.

La troisième stratégie consiste à combiner plusieurs représentations : tokens, identifiants d'événements, paramètres, embeddings sémantiques, encodages positionnels, vecteurs de comptage ou matrices de transition.

L'idée générale est que le prétraitement détermine fortement ce que le modèle pourra détecter. Un modèle entraîné uniquement sur des séquences d'identifiants d'événements détectera surtout des anomalies d'ordre ou de succession. En revanche, un modèle intégrant les paramètres, les fréquences et les temps d'arrivée pourra détecter des anomalies plus variées.

RQ4 — Quels types d'anomalies sont détectés et comment sont-elles identifiées ?

Les approches étudiées détectent principalement quatre types d'anomalies.

Les **outliers** sont des événements isolés qui ne ressemblent pas aux autres événements du jeu de données. Ils peuvent être liés à des paramètres inhabituels, à des tokens rares ou à des moments d'occurrence atypiques.

Les **anomalies séquentielles** apparaissent lorsque l'ordre attendu des événements change. C'est le type le plus étudié, car beaucoup de jeux de données courants contiennent ce genre d'anomalies.

Les **anomalies de fréquence** apparaissent lorsque certains événements se produisent trop souvent ou trop rarement par rapport au comportement normal. Par exemple, si certains événements liés à l'ouverture et à la fermeture de fichiers deviennent incohérents, cela peut signaler un problème.

Les **anomalies statistiques** concernent des propriétés quantitatives plus générales, comme les temps entre événements, les motifs saisonniers ou les distributions temporelles.

Pour identifier ces anomalies, les modèles utilisent plusieurs sorties possibles : classification binaire, classification multi-classe, erreur de reconstruction, distance dans un espace vectoriel, distribution de probabilités ou prédiction du prochain événement. Ensuite, la décision se fait par étiquette, par seuil, ou par comparaison avec les événements les plus probables.

RQ5 — Comment les modèles proposés sont-ils évalués ?

Les évaluations reposent principalement sur quelques jeux de données publics, notamment **HDFS**, **BGL**, **Thunderbird** et **OpenStack**.

Les auteurs critiquent implicitement cette situation : si presque toutes les approches sont testées sur les mêmes jeux de données, alors les conclusions risquent d’être trop dépendantes de ces données. Or ces jeux contiennent surtout certaines formes d’anomalies, notamment des anomalies séquentielles. Cela peut expliquer pourquoi les RNN sont si dominants.

Les métriques les plus utilisées sont la précision, le rappel, le F1-score, le taux de faux positifs et parfois l’accuracy. Toutefois, les auteurs notent que ces métriques peuvent être insuffisantes lorsque les données sont fortement déséquilibrées, ce qui est presque toujours le cas en détection d’anomalies.

Ils recommandent donc d’utiliser aussi des métriques plus robustes face au déséquilibre des classes, comme la spécificité ou le taux de vrais négatifs.

Un autre point critique est que les évaluations détectent souvent une séquence entière comme normale ou anormale. Or, dans une longue séquence, seuls quelques événements peuvent réellement être anormaux. Les auteurs recommandent donc de développer des évaluations capables d’identifier précisément les événements fautifs à l’intérieur d’une séquence.

RQ6 — Dans quelle mesure les approches dépendent-elles de données étiquetées et soutiennent-elles l’apprentissage incrémental ?

La détection d’anomalies dans les logs vise souvent à identifier des comportements inattendus, comme des pannes ou des cyberattaques. Par définition, ces événements ne sont pas toujours connus à l’avance. Les approches non supervisées ou semi-supervisées sont donc plus adaptées aux cas réels.

Pourtant, la revue montre que beaucoup d’approches utilisent encore des données étiquetées. Les auteurs recensent :

- **28 approches semi-supervisées ;**
- **8 approches non supervisées ;**
- **26 approches supervisées.**

Cette présence importante d’approches supervisées est surprenante, car elle suppose l’existence d’anomalies déjà connues et étiquetées.

Concernant l’apprentissage incrémental, le constat est encore plus critique : **54 approches sur 62** reposent uniquement sur un entraînement hors ligne. Seules **8 approches** permettent une adaptation continue du modèle, par réentraînement ou par des architectures capables d’évoluer avec les données.

RQ7 — Dans quelle mesure les résultats sont-ils reproductibles ?

La reproductibilité est partielle.

Du côté des données, la situation est relativement bonne : la majorité des publications utilisent au moins un jeu de données public. Les auteurs indiquent que seules quelques publications s’appuient exclusivement sur des données privées.

Mais du côté du code source, la situation est faible. Les auteurs trouvent peu de codes publiés par les auteurs originaux, ainsi que quelques réimplémentations. Cela signifie que les résultats sont difficilement reproductibles sans réécrire soi-même les approches à partir des descriptions des articles.

Les auteurs encouragent donc fortement les chercheurs à publier :

- leur code source ;
- leurs scripts d’expérience ;
- leurs jeux de données ;
- leurs paramètres d’entraînement ;
- leurs méthodes de prétraitement.

Sans cela, les comparaisons quantitatives à grande échelle restent fragiles.

Recommandations des auteurs

À partir de leurs réponses aux questions de recherche, les auteurs proposent plusieurs orientations pour les travaux futurs.

Premièrement, il faut créer ou identifier de nouveaux jeux de données de logs plus variés. Les données actuelles favorisent trop les anomalies séquentielles et ne

représentent pas suffisamment la diversité des anomalies possibles dans les systèmes réels.

Deuxièmement, il faut améliorer l'explicabilité des modèles. Les systèmes de détection doivent pouvoir indiquer non seulement qu'une anomalie existe, mais aussi pourquoi elle a été détectée.

Troisièmement, les recherches futures devraient considérer d'autres formes d'artefacts que les seules séquences d'événements : paramètres, distributions, corrélations, périodicité, valeurs numériques et relations entre événements.

Quatrièmement, il faut proposer des méthodes mieux adaptées aux contraintes pratiques : traitement en flux, apprentissage incrémental, adaptation dynamique, faible consommation de ressources et entraînement efficace.

Cinquièmement, les auteurs recommandent de localiser précisément les anomalies dans les séquences, au lieu de classer toute une séquence comme anormale.

Enfin, ils demandent une meilleure ouverture scientifique : publication du code, des données et des expériences reproductibles.

VI. Conclusion

L'article présente une revue de **62 approches scientifiques** portant sur la détection d'événements ou de processus anormaux dans les données de logs système à l'aide de l'apprentissage profond.

La revue montre que plusieurs architectures peuvent être utilisées :

- les RNN et CNN pour les données séquentielles ;
- les Transformers pour les représentations proches du langage ;
- les autoencodeurs et GAN pour les approches non supervisées ;
- d'autres modèles plus rares comme les GNN, MLP et EGNN.

Les caractéristiques utilisées pour l'entraînement et la détection sont également variées : séquences d'événements, comptages d'événements, tokens, paramètres, statistiques ou valeurs dérivées des logs.

Pour être traitées par des réseaux neuronaux, ces caractéristiques doivent être encodées sous forme numérique, par exemple avec des vecteurs sémantiques, des embeddings ou du one-hot encoding.

Les anomalies sont ensuite détectées soit directement par classification, soit indirectement à travers un score d'anomalie permettant de distinguer le comportement normal du comportement anormal.

La conclusion principale est que le domaine reste encore ouvert. Les défis les plus importants sont :

- le dépassement des seules anomalies séquentielles ;
- le manque d'explicabilité des modèles ;
- l'absence de jeux de données suffisamment représentatifs ;
- la faible reproductibilité liée au manque de code source public ;
- la difficulté de l'apprentissage incrémental et du traitement en flux.

En résumé, l'apprentissage profond offre un potentiel important pour la détection d'anomalies dans les logs, mais les approches actuelles restent encore trop dépendantes de jeux de données limités, de scénarios expérimentaux restreints et de modèles difficiles à expliquer.